UNIVERSIDAD DEL VALLE DE GUATEMALA

PROGRAMACIÓN ORIENTADA A OBJETOS

DIEGO ISAÍAS MARROQUÍN MOSCOSO

EJERCICIO #6

Polimorfismo vía interfaces

Análisis y diseño

Sistema central para administrar integraciones externas de consulta, pago, logística y facturación mediante herencia, interfaces, polimorfismo, sobrecarga, ordenamiento y el patrón MVC.

1. ¿Qué problema resuelve el programa?

El programa centraliza la comunicación con servicios externos de una plataforma de comercio electrónico. Mantiene un catálogo común para APIs de consulta, pasarelas de tarjeta, transferencias bancarias, billeteras digitales, logística y facturación FEL. Desde un menú de consola se pueden registrar y consultar integraciones, conectarlas, ejecutar sus operaciones, procesar cobros o reintegros, cotizar y rastrear envíos, generar documentos y conservar auditorías. El diseño evita que el flujo principal tenga que conocer cada proveedor concreto cuando la operación puede expresarse mediante una interfaz.

2. ¿Qué clases e interfaces representan el problema?

El diseño contiene once clases y tres interfaces. IntegracionAPI es la superclase abstracta. APIConsulta, PagoTarjeta, TransferenciaBancaria, BilleteraDigital, APILogistica y APIFacturacion son especializaciones concretas. Transaccional, Documentable y Auditable expresan capacidades independientes. CatalogoIntegraciones conserva la lista polimórfica y ofrece operaciones comunes; ControladorIntegraciones coordina el menú; VistaConsola gestiona entrada y salida; Main crea la carga inicial e inicia la aplicación.

3. ¿Qué información debe conservarse?

Toda integración conserva ID, nombre, URL base, API key, estado de conexión y latencia promedio. Las APIs de consulta guardan el tipo de consulta. Los tres medios de pago conservan su proveedor o marca, moneda y comisión. Logística conserva zona de cobertura, moneda y costo base. Facturación conserva NIT del emisor, establecimiento e historial. El catálogo almacena las referencias de las diez integraciones en una única ArrayList<IntegracionAPI>.

4. ¿Qué tipos y visibilidad tienen los datos?

Los identificadores, nombres, URLs, claves, monedas, proveedores, guías, formatos e historiales son String. La conexión es boolean. Latencias, comisiones, montos, pesos y costos son double. Los atributos comunes de IntegracionAPI son protected para que las subclases implementen su comportamiento. Los atributos particulares, el catálogo, la vista y el Scanner son private. Los constructores, getters, setters y operaciones son public; leerNumeroPositivo() es private y main() es static.

5. ¿Cómo se organiza la lista polimórfica?

CatalogoIntegraciones declara una sola ArrayList<IntegracionAPI>. En ella conviven objetos de las seis subclases, por lo que listar, buscar, conectar, desconectar, ejecutar la operación principal y ordenar no requieren listas separadas por categoría. agregarIntegracion(IntegracionAPI) acepta cualquier subtipo y equals() considera iguales dos objetos con el mismo ID, lo que evita duplicados.

6. ¿Cómo se aplica el polimorfismo vía herencia?

IntegracionAPI declara ejecutarOperacion() como método abstracto. Cada subclase sobrescribe exactamente esa firma. Cuando el controlador conserva una referencia IntegracionAPI y llama api.ejecutarOperacion(), Java selecciona la implementación del objeto real. También se sobrescribe toString() para mostrar datos comunes y particulares, y compareTo() permite ordenar referencias de cualquier subtipo por latencia.

7. ¿Cómo se aplica el polimorfismo vía interfaces?

Transaccional unifica cobros y reintegros de PagoTarjeta, TransferenciaBancaria y BilleteraDigital. Documentable permite generar documentos con APILogistica y APIFacturacion. Auditable permite registrar y consultar operaciones de APIFacturacion. El controlador y el catálogo comprueban la capacidad con instanceof y luego trabajan con una referencia de interfaz. De esa forma, el parámetro describe la capacidad requerida y no el proveedor concreto.

8. ¿Cómo funcionan los cobros y reintegros?

Cada medio transaccional valida que el monto sea finito y mayor que cero. Si la API está desconectada devuelve false. Al cobrar calcula monto + monto × comisión y muestra la moneda y el proveedor; el reintegro conserva el monto recibido. CatalogoIntegraciones.procesarReintegro() recibe IntegracionAPI, comprueba si también es Transaccional y delega la operación. Si no lo es, devuelve false sin lanzar una excepción de conversión.

9. ¿Cómo funcionan las operaciones logísticas?

APILogistica cotiza con costoBase + peso × 5, siempre que el peso sea válido y la API esté conectada. El rastreo devuelve un estado simulado asociado a la guía. Como implementación adicional, cada proveedor conserva su moneda: Cargo Expreso usa GTQ y DHL usa USD, evitando imprimir un símbolo fijo para todas las cotizaciones. La clase también genera una guía en un formato solicitado y usa TXT como formato predeterminado.

10. ¿Cómo funcionan la facturación, los documentos y la auditoría?

APIFacturacion genera una representación XML o TXT; cualquier otro formato devuelve “Formato no soportado”. certificarFactura() valida los datos, exige conexión y registra automáticamente la certificación. registrarOperacion() acumula líneas en historial y obtenerHistorial() las consulta. La

interfaz Documentable también es implementada por APILogistica, mientras Auditable se aplica a facturación.

11. ¿Cómo se usa la sobrecarga?

Documentable declara generarDocumento() y generarDocumento(String formato). CatalogoIntegraciones repite esa sobrecarga con parámetros de tipo Documentable: una versión solicita el formato predeterminado del objeto y la otra transmite el formato elegido por el usuario. APIFacturacion usa XML por defecto y APILogistica usa TXT. La sobrecarga mantiene una sola intención con dos formas de invocación.

12. ¿Cómo se ordena y compara el catálogo?

IntegracionAPI implementa Comparable<IntegracionAPI>. compareTo() usa Double.compare() sobre latenciaPromedio y Collections.sort() ordena la lista de menor a mayor latencia. equals() y hashCode() se basan en id, por lo que contains() detecta una integración duplicada aunque sus demás atributos sean diferentes.

13. ¿Cómo se administran conexión y estado?

Todas las integraciones comienzan desconectadas. conectar() y desconectar() actualizan el estado, y consultarEstado() devuelve “Conectada” o “Desconectada”. Las operaciones concretas revisan la conexión antes de simular un servicio externo. La API key se conserva para representar la configuración, pero toString() la oculta con asteriscos al mostrar detalles.

14. ¿Cómo se distribuyen las responsabilidades de MVC?

El modelo está compuesto por IntegracionAPI, sus subclases, las interfaces y CatalogoIntegraciones; allí se conservan datos y reglas. VistaConsola presenta el menú, lee entradas y muestra resultados. ControladorIntegraciones interpreta las catorce opciones, valida el tipo de API

y coordina modelo y vista. Main solamente construye los objetos, carga los datos e inicia el controlador.

15. ¿Cómo se inicializa el sistema?

Main crea CatalogoIntegraciones y llama cargarDatosIniciales(), que registra diez APIs con IDs API001 a API010. Luego crea VistaConsola, inyecta catálogo y vista en ControladorIntegraciones y llama iniciar(). La carga incluye dos consultas, dos tarjetas, dos transferencias, una billetera, dos proveedores logísticos y una API de facturación.

16. ¿Qué validaciones y decisiones de encapsulación utiliza el programa?

Los constructores y setters rechazan textos nulos o vacíos, números NaN o infinitos, latencias y costos negativos, comisiones fuera de 0 a 1, y montos o pesos no positivos. El controlador convierte entradas numéricas y captura IllegalArgumentException para mantener el menú activo. Los atributos están protegidos o privados y se consultan mediante getters. obtenerIntegraciones() devuelve actualmente la lista interna, por lo que el código consumidor debe evitar modificarla directamente; una vista inmutable sería una posible mejora futura.

17. ¿Cómo se puede ampliar el diseño?

Un proveedor nuevo puede extender IntegracionAPI, implementar ejecutarOperacion() y añadirse al catálogo. Si procesa dinero, documentos o auditoría, implementa la interfaz correspondiente y funciona con las opciones generales sin reescribir el catálogo. Un nuevo medio transaccional funcionaría con cobro y reintegro; un nuevo Documentable funcionaría con la generación sobrecargada. Las operaciones exclusivas de logística y facturación todavía dependen de las clases concretas APILogistica y APIFacturacion.

18. ¿Qué elementos adicionales se implementaron respecto del enunciado?

Además de los contratos requeridos, el programa incorpora conexión y desconexión explícitas, enmascaramiento de la API key, validaciones numéricas completas, rechazo de IDs duplicados, historial acumulativo de auditoría, certificación simulada, dos formatos de factura, rastreo de paquetes, monedas específicas para proveedores logísticos, marca de tarjeta diferenciada, catorce opciones de administración y mensajes detallados para operaciones rechazadas.

Requisitos funcionales del sistema

• Mantener una única lista polimórfica con al menos diez integraciones de naturaleza diversa.

• Conservar los datos comunes en IntegracionAPI y los particulares en cada subclase.

• Listar todas las integraciones con ID, nombre y estado de conexión.

• Buscar una API por ID y mostrar sus parámetros sin revelar la API key.

• Ordenar el catálogo por latencia promedio mediante Comparable.

• Conectar, desconectar y ejecutar la operación principal de cualquier integración.

• Procesar cobros y reintegros mediante Transaccional.

• Rechazar un reintegro solicitado a una API incompatible sin provocar una excepción de tipo.

• Cotizar envíos por peso y mostrar el costo en la moneda del proveedor.

• Rastrear paquetes por número de guía.

• Generar documentos mediante Documentable con formato predeterminado o solicitado.

• Certificar facturas y registrar esa operación en el historial.

• Registrar y consultar auditorías mediante Auditable.

• Validar textos, montos, pesos, costos, comisiones y latencias.

• Mantener el menú de consola activo hasta seleccionar la opción cero.

• Separar modelo, vista y controlador de acuerdo con MVC.

Carga inicial de integraciones

Todas las APIs comienzan desconectadas. El orden siguiente corresponde a la carga escrita en CatalogoIntegraciones.cargarDatosIniciales().

1. API001, Consulta de Estado: APIConsulta; latencia 120 ms; consulta ESTADO.
2. API002, Consulta de Latencia: APIConsulta; latencia 90 ms; consulta LATENCIA.
3. API003, Visa: PagoTarjeta; latencia 150 ms; marca Visa; GTQ; comisión 5%.
4. API004, Mastercard: PagoTarjeta; latencia 180 ms; marca Mastercard; GTQ; comisión 4%.
5. API005, Banco Industrial: TransferenciaBancaria; latencia 200 ms; GTQ; comisión 2%.
6. API006, Banrural: TransferenciaBancaria; latencia 170 ms; GTQ; comisión 3%.
7. API007, PayPal: BilleteraDigital; latencia 130 ms; USD; comisión 5%.
8. API008, Cargo Expreso: APILogistica; latencia 110 ms; Guatemala; GTQ; costo base 25.
9. API009, DHL: APILogistica; latencia 160 ms; Internacional; USD; costo base 75.
10. API010, Certificador FEL: APIFacturacion; latencia 140 ms; NIT 1234567-8; Tienda Central.

Descripción de las clases e interfaces

Notación: + público; - privado; # protegido. {abstract} identifica una clase u operación abstracta, <<interface>> una interfaz y {static} el punto de entrada. Los miembros heredados no se repiten en cada subclase.

IntegracionAPI {abstract}

Propósito de la clase:

Superclase abstracta que concentra configuración, conexión, comparación, identidad y el contrato de operación principal.

Propósito de los atributos:

id: String. Identificador único.

nombre: String. Nombre visible.

urlBase: String. Dirección del servicio.

apiKey: String. Credencial almacenada y enmascarada al mostrar.

conectada: boolean. Estado de conexión.

latenciaPromedio: double. Milisegundos usados para ordenar.

Propósito de los métodos:

+ IntegracionAPI(id, nombre, urlBase, apikey, latenciaPromedio). Valida e inicializa los datos comunes.

+ getters y setters. Consultan o actualizan los atributos permitidos con validación.

+ ejecutarOperacion(): String {abstract}. Contrato implementado por todas las subclases.

+ conectar(): void y desconectar(): void. Cambian el estado.

+ consultarEstado(): String. Devuelve Conectada o Desconectada.

+ compareTo(otra: IntegracionAPI): int. Compara latencias.

+ equals(obj: Object): boolean y hashCode(): int. Definen identidad por ID.

+ toString(): String. Muestra configuración común y oculta la clave.

Transaccional <<interface>>

Propósito de la interfaz:

Contrato para componentes que procesan movimientos financieros.

Propósito de los atributos:

No declara atributos propios.

Propósito de los métodos:

+ procesarCobro(monto: double): boolean. Intenta cobrar un monto válido.

+ procesarReintegro(monto: double): boolean. Intenta devolver un monto válido.

Documentable <<interface>>

Propósito de la interfaz:

Contrato sobrecargado para componentes capaces de producir documentos.

Propósito de los atributos:

No declara atributos propios.

Propósito de los métodos:

+ generarDocumento(): String. Usa el formato predeterminado.

+ generarDocumento(formato: String): String. Usa el formato solicitado.

Auditable <<interface>>

Propósito de la interfaz:

Contrato para registrar operaciones y consultar su historial.

Propósito de los atributos:

No declara atributos propios.

Propósito de los métodos:

+ registrarOperacion(operacion: String): void. Añade una operación válida.

+ obtenerHistorial(): String. Devuelve el registro acumulado.

APIConsulta

Propósito de la clase:

Especialización para consultar estado, disponibilidad o latencia.

Propósito de los atributos:

- tipoConsulta: String. Selecciona ESTADO, DISPONIBILIDAD o LATENCIA.

Propósito de los métodos:

+ APIConsulta(..., tipoConsulta). Inicializa datos comunes y valida el tipo.

+ getTipoConsulta() y setTipoConsulta(tipo). Consultan o actualizan la selección.

+ consultarInformacion(): String. Ejecuta la consulta configurada.

+ ejecutarOperacion(): String. Delega en consultarInformacion().

+ toString(): String. Añade el tipo de consulta.

PagoTarjeta

Propósito de la clase:

Pasarela transaccional que procesa operaciones con una marca de tarjeta.

Propósito de los atributos:

- marcaTarjeta: String. Visa, Mastercard u otra marca.

- moneda: String. Moneda de la operación.

- comision: double. Proporción entre 0 y 1.

Propósito de los métodos:

+ PagoTarjeta(..., marcaTarjeta, moneda, comision). Valida e inicializa.

+ getters y setters. Administran marca, moneda y comisión.

+ procesarCobro(monto): boolean. Calcula monto más comisión.

+ procesarReintegro(monto): boolean. Simula la devolución.

+ ejecutarOperacion(): String. Informa disponibilidad transaccional.

+ toString(): String. Añade marca, moneda y comisión.

TransferenciaBancaria

Propósito de la clase:

Integración transaccional para cobros y reintegros mediante una entidad bancaria.

Propósito de los atributos:

- banco: String. Entidad que procesa la transferencia.

- moneda: String. Moneda de operación.

- comision: double. Proporción aplicada al cobro.

Propósito de los métodos:

+ TransferenciaBancaria(..., banco, moneda, comision). Inicializa y valida.

+ getters y setters. Administran banco, moneda y comisión.

+ procesarCobro(monto): boolean. Calcula monto total con comisión.

+ procesarReintegro(monto): boolean. Simula el reintegro bancario.

+ ejecutarOperacion(): String. Informa disponibilidad.

+ toString(): String. Añade banco, moneda y comisión.

BilleteraDigital

Propósito de la clase:

Integración transaccional para pagos por medio de una billetera digital.

Propósito de los atributos:

- proveedorBilletera: String. Proveedor del servicio.

- moneda: String. Moneda de operación.

- comision: double. Proporción aplicada.

Propósito de los métodos:

+ BilleteraDigital(..., proveedorBilletera, moneda, comision). Inicializa y valida.

+ getters y setters. Administran proveedor, moneda y comisión.

+ procesarCobro(monto): boolean. Calcula monto total con comisión.

+ procesarReintegro(monto): boolean. Simula la devolución.

+ ejecutarOperacion(): String. Informa disponibilidad.

+ toString(): String. Añade proveedor, moneda y comisión.

APILogistica

Propósito de la clase:

Proveedor logístico que cotiza, rastrea paquetes y genera guías.

Propósito de los atributos:

- zonaCobertura: String. Área atendida.

- moneda: String. Moneda de la cotización.

- costoBase: double. Valor fijo antes del peso.

Propósito de los métodos:

+ APILogistica(..., zonaCobertura, moneda, costoBase). Inicializa y valida.

+ getters y setters. Administran zona, moneda y costo.

+ cotizarEnvio(peso: double): double. Devuelve costoBase + peso × 5 o NaN si está desconectada.

+ rastrearPaquete(guia: String): String. Devuelve un estado simulado.

+ generarDocumento(formato: String): String. Genera una guía.

+ generarDocumento(): String. Usa TXT.

+ ejecutarOperacion(): String. Informa disponibilidad logística.

+ toString(): String. Añade zona, moneda, costo y tarifa.

APIFacturacion

Propósito de la clase:

Certificador FEL que genera facturas y conserva auditorías.

Propósito de los atributos:

- nitEmisor: String. Identificación fiscal.

- establecimiento: String. Sede emisora.

- historial: String. Registro acumulado de operaciones.

Propósito de los métodos:

+ APIFacturacion(..., nitEmisor, establecimiento). Inicializa datos e historial.

+ getters y setters. Administran NIT y establecimiento.

+ certificarFactura(datos: String): String. Certifica y registra la operación.

+ generarDocumento(formato: String): String. Produce XML o TXT.

+ generarDocumento(): String. Usa XML.

+ ejecutarOperacion(): String. Informa disponibilidad FEL.

+ obtenerHistorial(): String. Devuelve el registro.

+ registrarOperacion(operacion: String): void. Acumula una línea.

+ toString(): String. Añade configuración e historial.

CatalogoIntegraciones

Propósito de la clase:

Modelo central que conserva la única lista polimórfica, carga datos y delega capacidades comunes.

Propósito de los atributos:

- integraciones: ArrayList<IntegracionAPI>. Registro único de APIs.

Propósito de los métodos:

+ CatalogoIntegraciones(). Crea la lista vacía.

+ agregarIntegracion(integracion: IntegracionAPI): void. Rechaza null e IDs duplicados.

+ buscarIntegracion(id: String): IntegracionAPI. Devuelve coincidencia o null.

+ obtenerIntegraciones(): List<IntegracionAPI>. Devuelve la lista registrada.

+ ordenarPorLatencia(): void. Aplica Collections.sort().

+ cargarDatosIniciales(): void. Registra las diez APIs.

+ procesarReintegro(integracion: IntegracionAPI, monto: double): boolean. Delega si es Transaccional.

+ generarDocumento(api: Documentable): String. Usa formato predeterminado.

+ generarDocumento(api: Documentable, formato: String): String. Sobrecarga con formato.

+ auditar(api: Auditable, operacion: String): void. Delega el registro.

ControladorIntegraciones

Propósito de la clase:

Coordina las opciones de la consola entre el catálogo y la vista.

Propósito de los atributos:

- catalogo: CatalogoIntegraciones. Referencia al modelo.

- vista: VistaConsola. Referencia a la interfaz de consola.

Propósito de los métodos:

+ ControladorIntegraciones(catalogo, vista). Valida y conserva referencias.

+ iniciar(): void. Mantiene el ciclo del menú hasta la opción cero.

+ listarIntegraciones(): void. Solicita la lista y la entrega a la vista.

+ buscarIntegracion(): void. Lee el ID y muestra el detalle.

+ ordenarIntegraciones(): void. Ordena y muestra el catálogo.

+ ejecutarOpcion(opcion: int): void. Atiende las catorce operaciones.

- leerNumeroPositivo(mensaje: String): double. Convierte y valida entradas numéricas.

VistaConsola

Propósito de la clase:

Gestiona toda la entrada y salida de la aplicación por consola.

Propósito de los atributos:

- scanner: Scanner. Lector encapsulado de System.in.

Propósito de los métodos:

+ VistaConsola(). Inicializa Scanner.

+ mostrarMenu(): void. Imprime las catorce opciones y salir.

+ leerOpcion(): int. Convierte una opción o devuelve -1 si es inválida.

+ leerTexto(mensaje: String): String. Solicita y limpia texto.

+ mostrarIntegraciones(lista: List<IntegracionAPI>): void. Imprime tabla resumida.

+ mostrarDetalle(api: IntegracionAPI): void. Usa toString() polimórfico.

+ mostrarMensaje(mensaje: String): void. Imprime un resultado.

Main

Propósito de la clase:

Punto de entrada que construye el catálogo, carga las diez APIs, crea vista y controlador e inicia el sistema.

Propósito de los atributos:

No declara atributos propios.

Propósito de los métodos:

+ main(args: String[]): void {static}. Ensambla los componentes y llama controlador.iniciar().

Relaciones y decisiones de diseño

• APIConsulta, PagoTarjeta, TransferenciaBancaria, BilleteraDigital, APILogistica y APIFacturacion heredan de IntegracionAPI.

• PagoTarjeta, TransferenciaBancaria y BilleteraDigital realizan Transaccional; APILogistica y APIFacturacion realizan Documentable; APIFacturacion también realiza Auditable.

• CatalogoIntegraciones conserva una asociación navegable hacia 0..* IntegracionAPI. La lista usa el tipo base para admitir objetos heterogéneos.

• CatalogoIntegraciones depende de las interfaces en sus parámetros, lo que expresa la capacidad requerida y mantiene bajo acoplamiento.

• ControladorIntegraciones conserva asociaciones hacia CatalogoIntegraciones y VistaConsola. Main depende de las clases concretas que construye.

• El uso de instanceof se limita a comprobar capacidades o funciones exclusivas; ejecutarOperacion() y toString() usan despacho polimórfico real.

• La API key se oculta en toString(); la moneda logística evita asumir que todos los proveedores cotizan en quetzales.

• La lista expuesta por obtenerIntegraciones() sigue siendo modificable; devolver Collections.unmodifiableList() sería una mejora de encapsulación, no una función actualmente implementada.

Verificación del programa

Se compilaron los catorce archivos Java con codificación UTF-8 y advertencias activadas. Se ejecutó una batería automática de 52 comprobaciones y todas finalizaron satisfactoriamente. También se recorrieron opciones del menú para comprobar nombres, conexión, ejecución, rechazo seguro de capacidades y ordenamiento.

Carga, búsqueda y ordenamiento

Se obtuvieron diez integraciones. API001, API003, API005, API007, API008 y API010 coincidieron con sus tipos esperados; un ID inexistente devolvió null. Todas iniciaron desconectadas. Después de ordenar, API002 quedó primero con 90 ms y API005 al final con 200 ms; cada par consecutivo cumplió el orden ascendente.

Operaciones transaccionales

PagoTarjeta desconectada devolvió false. Al conectarla, un cobro de 100 GTQ calculó 105 GTQ con 5% de comisión y un reintegro de 25 fue aceptado. Banco Industrial calculó 204 GTQ para un cobro de 200 con 2% y el catálogo delegó correctamente un reintegro de 50.

Reintegro incompatible

CatalogoIntegraciones.procesarReintegro() recibió APILogistica y devolvió false sin ClassCastException ni interrupción. Este resultado verifica la limitación esencial del enunciado.

Logística y monedas

Cargo Expreso desconectado devolvió NaN al cotizar. Conectado, un paquete de 2 kg produjo 25 + 2 × 5 = 35 GTQ. DHL produjo 75 + 2 × 5 = 85 USD. También se verificaron el rastreo por guía y la generación predeterminada TXT y solicitada XML.

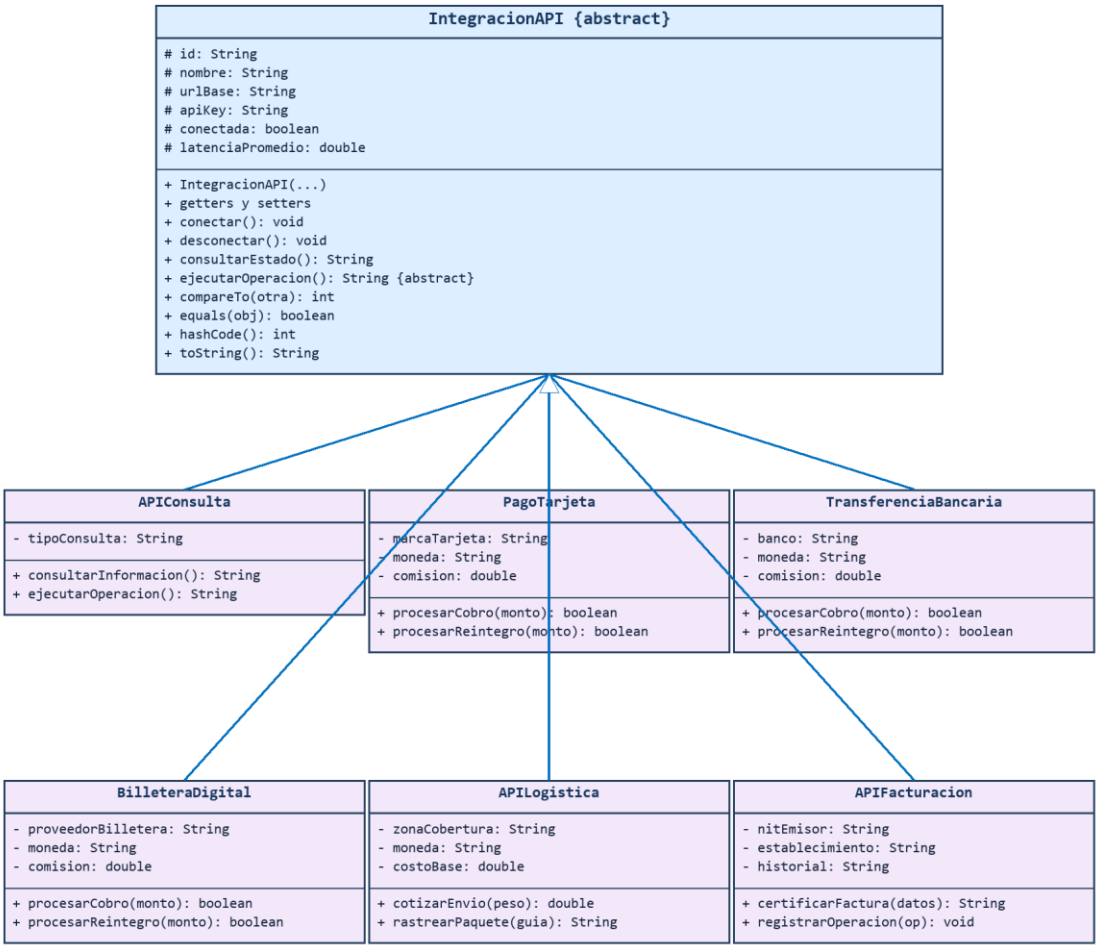
Facturación y auditoría

El formato predeterminado produjo XML; TXT generó la representación correspondiente y PDF devolvió “Formato no soportado”. Certificar “Venta 100” registró la operación en el historial. Una auditoría manual adicional también permaneció disponible al consultar el registro.

Validaciones e identidad

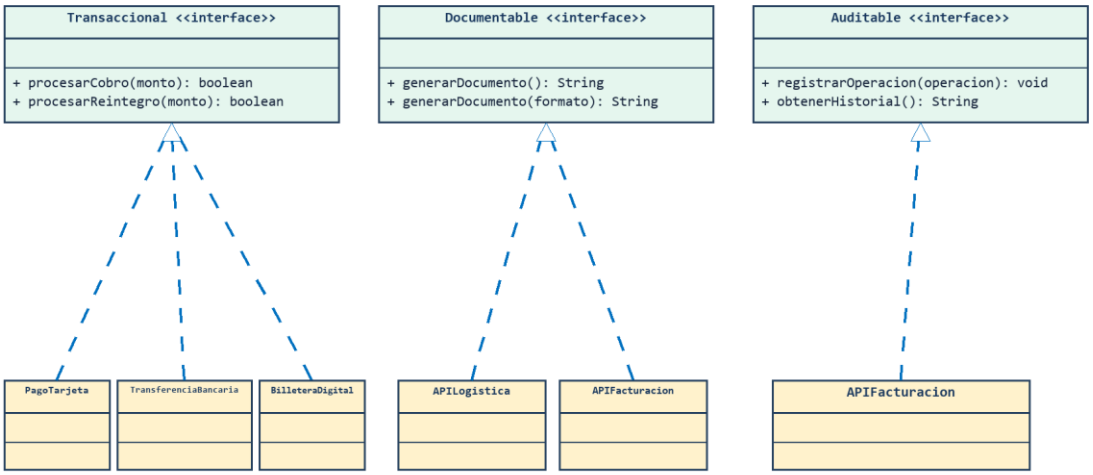
Una segunda integración con ID API001 fue rechazada aunque sus demás valores fueran distintos, confirmando equals(), hashCode() y la prevención de duplicados. Los casos de montos, pesos, comisiones, costos, latencias y textos inválidos quedan protegidos por las validaciones de constructores, setters y métodos.

Diseño UML de la jerarquía



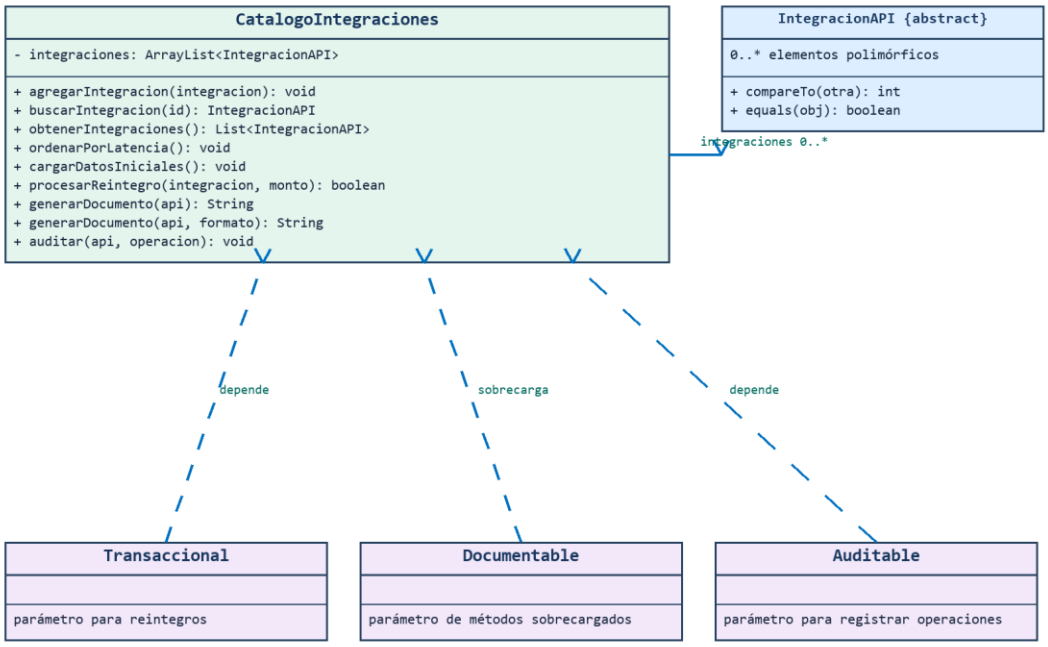
Las seis clases concretas generalizan hacia IntegracionAPI. La superclase concentra estado, comparación e identidad; cada subtipo conserva solo sus atributos y operaciones particulares.

Diseño UML de las interfaces



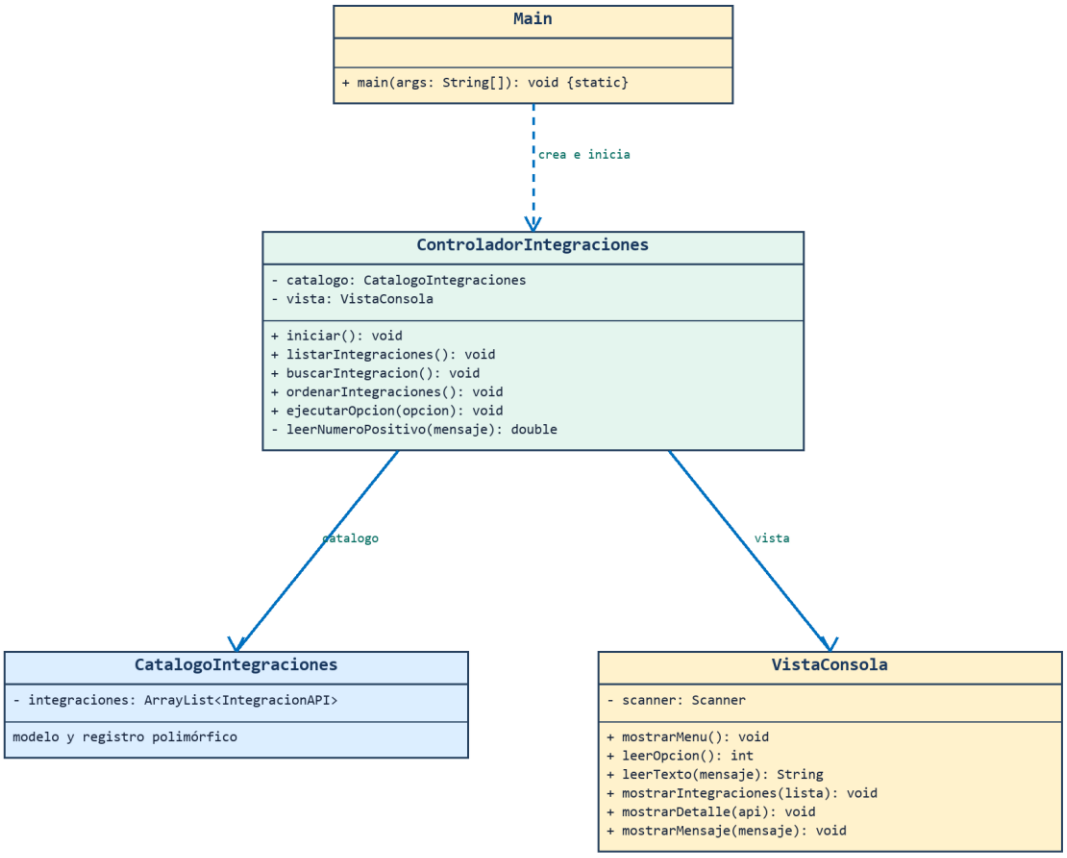
Las líneas discontinuas con triángulo vacío representan realizaciones. Los tres medios de pago implementan Transaccional; logística y facturación implementan Documentable; facturación también implementa Auditable.

Diseño UML del catálogo



CatalogoIntegraciones conserva 0..* referencias IntegracionAPI y usa parámetros de interfaz para reintegros, documentos y auditoría. Los dos métodos generarDocumento() evidencian la sobrecarga solicitada.

Diseño UML de MVC e inicio



Main crea e inicia el sistema. El controlador coordina el catálogo y la vista; CatalogoIntegraciones representa el registro del modelo y VistaConsola concentra la interacción con el usuario.